

COMPLETE SOFTWARE SPECIFICATION & SYSTEM MANUAL

Full-Stack System Architecture, API Reference & User Guide

An end-to-end technical documentation manual for LearnHub: modern full-stack MERN EdTech LMS featuring dynamic cataloging, JWT authentication, role-based administration, interactive video learning player, and real-time student analytics.

SYSTEM VERSION

v1.0.0 Production

ARCHITECTURE

MERN Stack (Vite + Node)

SECURITY TIER

JWT + BCrypt RBAC

DOCUMENTATION DATE

August 2026

Table of Contents

- | | |
|---|--|
| 01. Executive Overview & Platform Summary | 02. System Architecture & Ecosystem Stack |
| 03. Database Schema & Data Models | 04. RESTful API Reference Specification |
| 05. Security & Role-Based Access Control (RBAC) | 06. Comprehensive Visual UI & Workflow Walkthrough |
| 07. User Guides (Student, Instructor, Admin) | 08. Installation, Deployment & Operations Guide |

1 Executive Overview & Platform Summary

LearnHub is a next-generation, cloud-native Learning Management System (LMS) and e-learning platform designed to deliver an engaging, high-performance education experience for students, educators, and enterprise administrators.

For Students

Self-paced video classroom, multi-factor course filtering, direct enrollment, interactive curriculum progression, certificate tracking, and personal profile management.

For Instructors

Comprehensive course authoring tools, dynamic curriculum builder, lesson video embedding, enrolled student monitoring, and rating analytics.

For Platform Admins

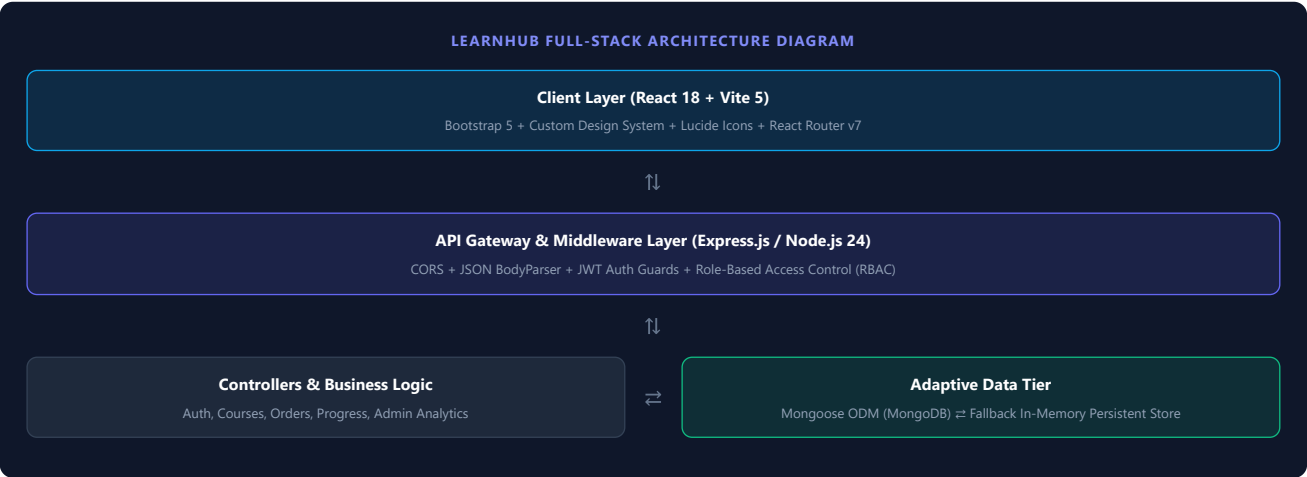
Centralized command center with real-time KPI metrics, global user management (roles & permissions), course publishing moderation, enrollment audits, and revenue reports.

Zero-Latency Fallback Architecture

LearnHub includes an automated dual-mode storage engine: when local MongoDB server instances are active, it connects seamlessly; in environments where MongoDB is offline, it activates an **Adaptive In-Memory Database Store** with JSON disk synchronization to guarantee 100% continuous uptime without configuration barriers.

2 System Architecture & Ecosystem Stack

LearnHub is structured following the modern **MERN** (MongoDB, Express.js, React.js, Node.js) architectural paradigm with client-side SPA routing via Vite and stateless JSON Web Token (JWT) API communication.



Technology Ecosystem Matrix

LAYER	TECHNOLOGY	VERSION	PRIMARY FUNCTION & RATIONALE
Frontend Framework	React.js	18.2.0	Component-driven virtual DOM rendering for responsive, interactive state management.
Build Tool & Bundler	Vite	5.1.6	Lightning-fast HMR (Hot Module Replacement) and optimized tree-shaken production bundles.
Styling & Design System	Bootstrap 5 + Custom CSS	5.3.3	Tailored design tokens, glassmorphism effects, modern gradients, and responsive grid layouts.
Routing System	React Router DOM	7.18.2	Client-side declarative routing with nested protected route wrappers for RBAC.
Backend Engine	Node.js & Express.js	Node 24 / Express 4.22	High-throughput asynchronous REST API server with modular controllers and routes.
Authentication	JWT & BCrypt.js	JWT 9.0 / BCrypt 3.0	Stateless cryptographic tokens with 7-day expiration and 10-round salted password hashing.
Persistence Tier	MongoDB / Mongoose	Mongoose 8.24	Schema-enforced NoSQL document database with fallback adaptive local memory store.

3 Database Schema & Data Models

The database architecture is designed with high normalization for transactional integrity combined with embedded sub-documents for nested curriculum components.

User Model (`models/User.js`)

```
{
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  phone: { type: String, default: '' },
  password: { type: String, required: true }, // Bcrypt hash
  role: { type: String, enum: ['student', 'instructor', 'admin'], default: 'student' },
  avatar: { type: String, default: 'avatar_url' },
  bio: { type: String, default: '' },
  enrolledCourses: [{ type: String }],
  wishlist: [{ type: String }],
  timestamps: true
}
```

Course Model (`models/Course.js`)

```
{
  id: { type: String, required: true, unique: true },
  title: { type: String, required: true },
  subtitle: { type: String },
  description: { type: String },
  category: { type: String, required: true },
  level: { type: String, enum: ['Beginner', 'Intermediate', 'Advanced'], default: 'Beginner' },
  price: { type: Number, required: true },
  originalPrice: { type: Number },
  rating: { type: Number, default: 4.8 },
  reviewsCount: { type: Number, default: 0 },
  hours: { type: Number },
  instructorName: { type: String },
  modules: [
    {
      id: String,
      title: String,
      lessons: [{ id: String, title: String, duration: String, videoUrl: String }],
    }
  ],
  status: { type: String, enum: ['published', 'draft', 'pending'] },
  timestamps: true
}
```

Order Model (`models/Order.js`)

```
{
  id: { type: String, required: true, unique: true },
  userId: { type: String, required: true, index: true },
  userName: { type: String, required: true },
  items: [
    {
      courseId: { type: String, required: true },
      title: { type: String, required: true },
      price: { type: Number, required: true }
    }
  ],
  totalAmount: { type: Number, required: true },
  paymentStatus: { type: String, enum: ['completed', 'pending', 'failed'], default: 'completed' },
  paymentMethod: { type: String, default: 'Online Enrollment' },
  timestamps: true
}
```

Progress Model (`models/Progress.js`)

```
{
  key: { type: String, required: true, unique: true },
  userId: { type: String, required: true, index: true },
  courseId: { type: String, required: true, index: true },
  completedLessons: [{ type: String }],
  percentage: { type: Number, default: 0 },
  lastWatchedLesson: { type: String, default: '' },
  certificateEarned: { type: Boolean, default: false },
  timestamps: true
}
```


4 RESTful API Reference Specification

The LearnHub backend provides a clean, predictable REST API that returns standardized JSON structures. All requests requiring authentication must pass the bearer token in the `Authorization: Bearer <token>` header.

Authentication & Profile Endpoints

METHOD	ENDPOINT	ACCESS LEVEL	DESCRIPTION & PAYLOAD
POST	/api/auth/register	PUBLIC	Registers a new student account. Body: { name, email, password, phone }
POST	/api/auth/login	PUBLIC	Authenticates user and returns JWT token & user profile. Body: { email, password }
GET	/api/auth/me	JWT AUTH	Fetches current authenticated user's session profile, wishlist, and enrollments.
PUT	/api/auth/profile	JWT AUTH	Updates user personal details (name, phone, bio, avatar URL).
POST	/api/auth/logout	PUBLIC	Invalidates client session tokens.

Course & Catalog Endpoints

METHOD	ENDPOINT	ACCESS LEVEL	DESCRIPTION & PAYLOAD
GET	/api/courses	PUBLIC	Returns list of all active courses with optional query parameters (category, level, search).
GET	/api/courses/:id	PUBLIC	Fetches deep details, syllabus modules, lessons, and instructor profile for a course ID.
POST	/api/courses/:id/enroll	JWT AUTH	Enrolls the authenticated student into the course and initializes progress record.
POST	/api/courses	ADMIN	Creates a new course with nested curriculum modules and lesson definitions.
DELETE	/api/courses/:id	ADMIN	Deletes course and removes it from user enrollment indices.

Learning Progress & Admin Endpoints

METHOD	ENDPOINT	ACCESS LEVEL	DESCRIPTION & PAYLOAD
GET	/api/progress/:courseId	JWT AUTH	Retrieves completed lesson IDs, percentage completed, and certificate issuance status.
POST	/api/progress/lesson-complete	JWT AUTH	Marks a lesson as completed, recalculates completion %, and updates last watched position.
GET	/api/admin/stats	ADMIN	Returns aggregate metrics: Total Revenue, Total Students, Active Courses, Completion Rates.
GET	/api/admin/users	ADMIN	Returns all registered platform users with roles, enrollment counts, and registration dates.
PUT	/api/admin/users/:userId/role	ADMIN	Elevates or adjusts user role (student, instructor, admin).
DELETE	/api/admin/users/:userId	ADMIN	Deactivates user record and clears session tokens.
GET	/api/admin/enrollments	ADMIN	Fetches platform-wide active student course enrollments.

5 Security & Role-Based Access Control (RBAC)

Security is implemented at both the network boundary and within the application layer through defense-in-depth methodologies.

JWT Bearer Tokens

Signed using high-entropy secrets, encapsulating user ID, email, and role claims. Tokens are verified on every protected API call via Express middleware.

BCrypt Password Security

Zero plain-text password storage. All user credentials undergo 10 rounds of cryptographic salting and hashing before persisting.

RBAC Route Guards

Double-layer route protection: Client-side `<ProtectedRoute>` components prevent unauthorized DOM views, while backend middleware rejects unauthorized API calls with 403 Forbidden.

Role Access Permission Matrix

PLATFORM CAPABILITY / RESOURCE	PUBLIC / GUEST	STUDENT	INSTRUCTOR	ADMINISTRATOR
Browse Course Catalog & Search	✓	✓	✓	✓
Course Enrollment	✗	✓	✓	✓
Watch Video Lessons & Mark Progress	✗	✓ (Enrolled)	✓	✓
Student Dashboard & Profile Management	✗	✓	✓	✓
Course Authoring & Curriculum Management	✗	✗	✓	✓
Admin Management Suite (Users, Reports, Analytics)	✗	✗	✗	✓

6 Comprehensive Visual UI & Workflow Walkthrough

The following section illustrates the primary validated interfaces, workflows, and functional views in the LearnHub ecosystem.

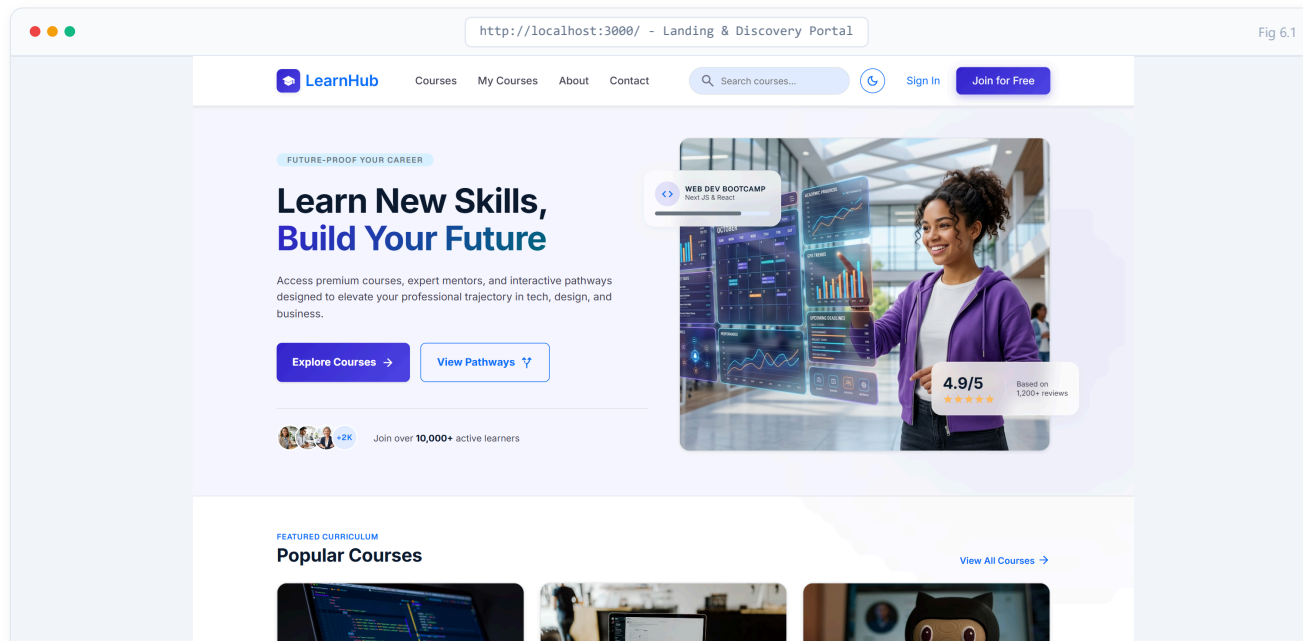


Figure 6.1: Modern Hero Showcase, Featured Course Highlights, Category Exploration, and Stats Bar.

Route: /

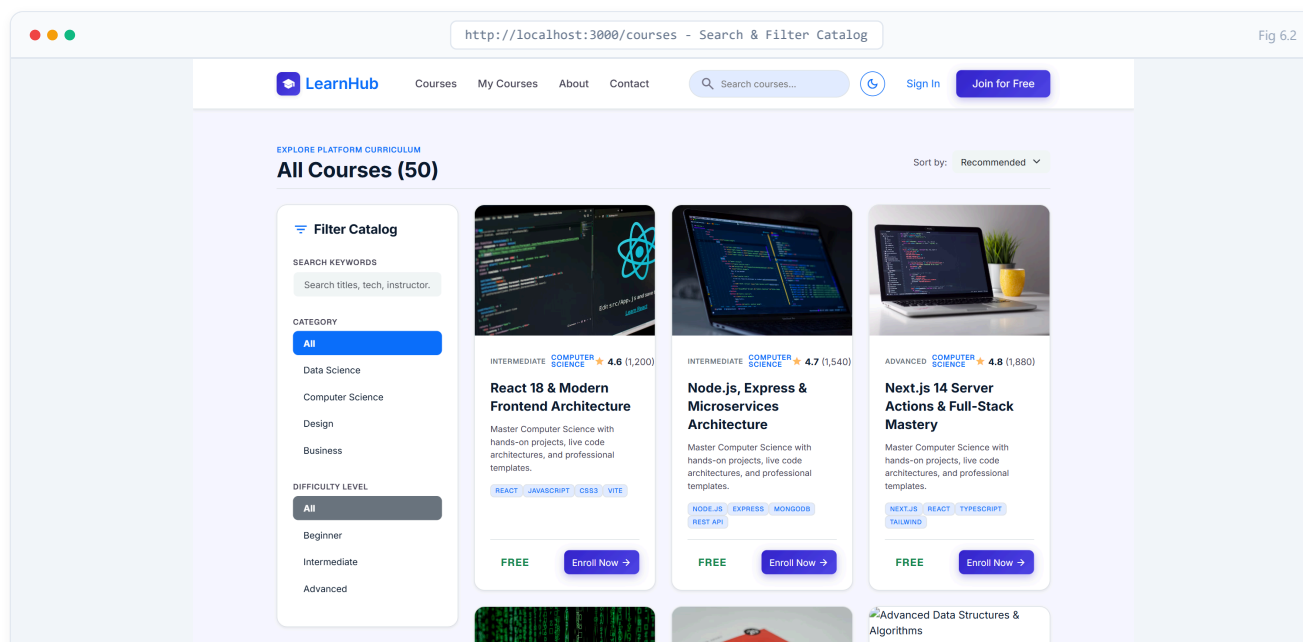
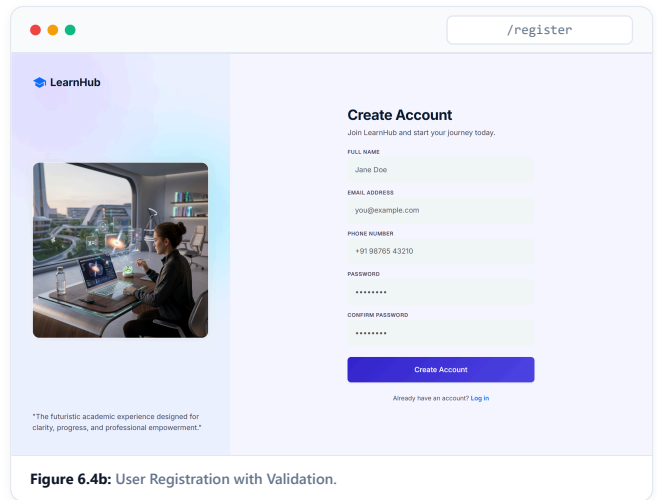
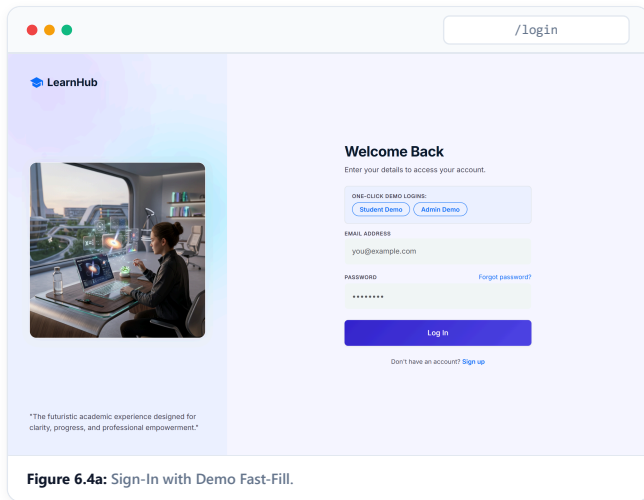
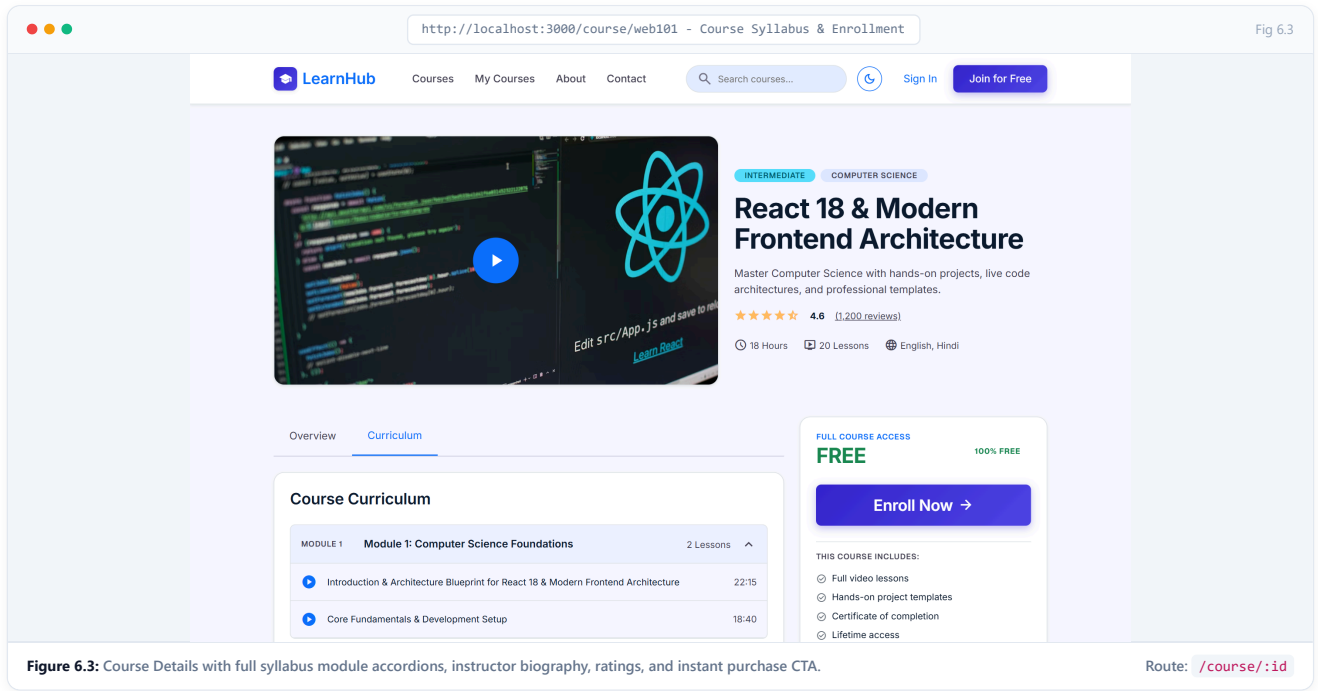


Figure 6.2: Dynamic Course Catalog with category sidebar filters, difficulty badges, search bar, and live pricing.

Route: /courses



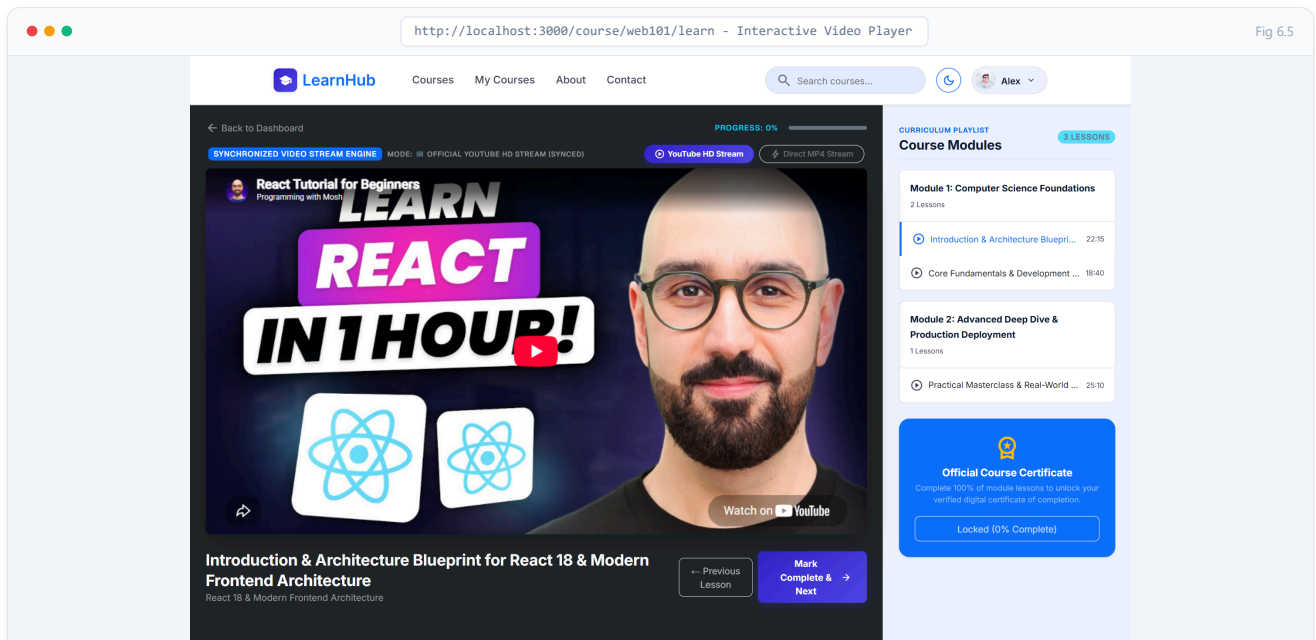


Figure 6.5: Interactive Video Learning Player with automated lesson progression, lesson playlist navigation, and dynamic checkmarking.

Route: `/course/:id/learn`

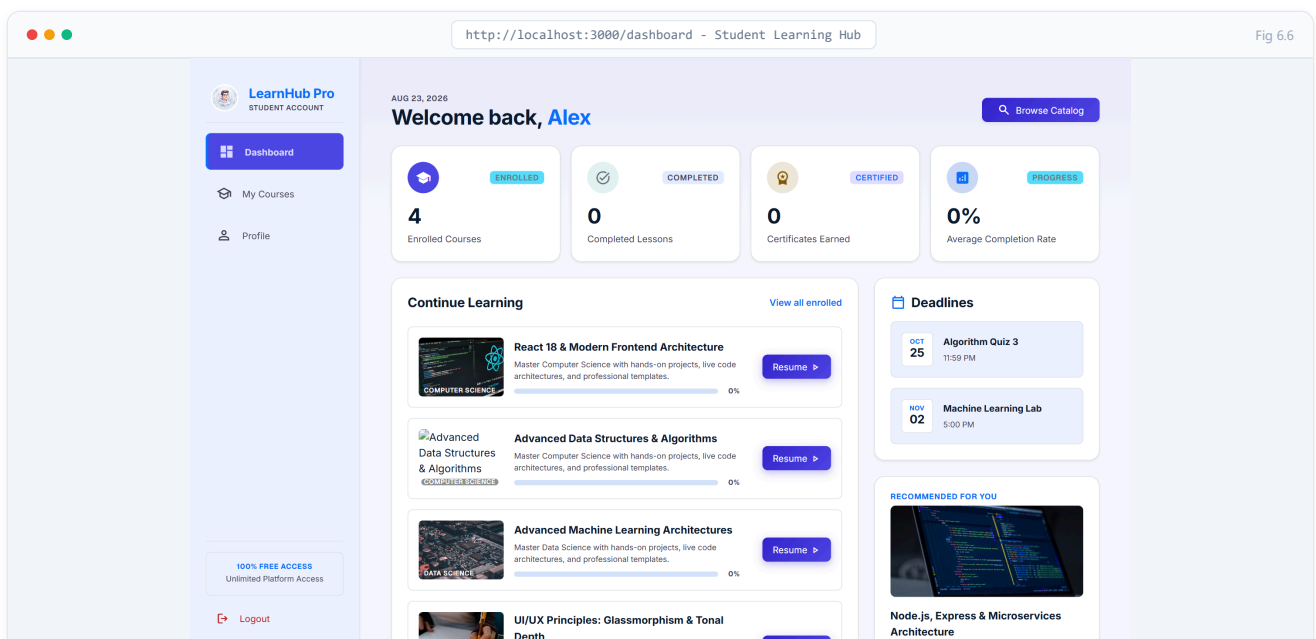


Figure 6.6: Student Dashboard showing active enrolled courses, overall completion percentages, study hours, and quick continue buttons.

Route: `/dashboard`

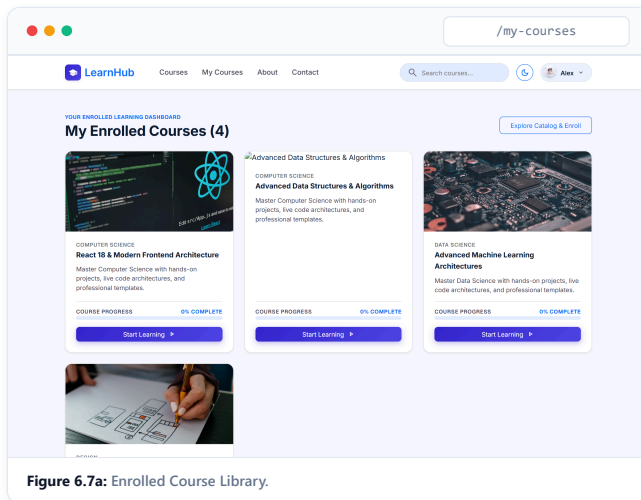


Figure 6.7a: Enrolled Course Library.

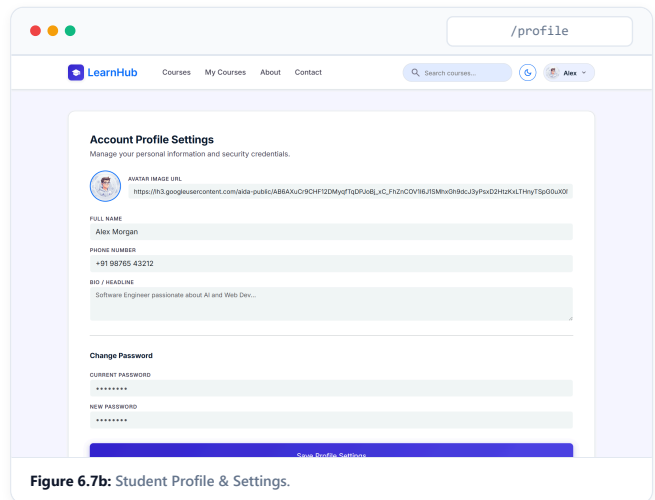


Figure 6.7b: Student Profile & Settings.

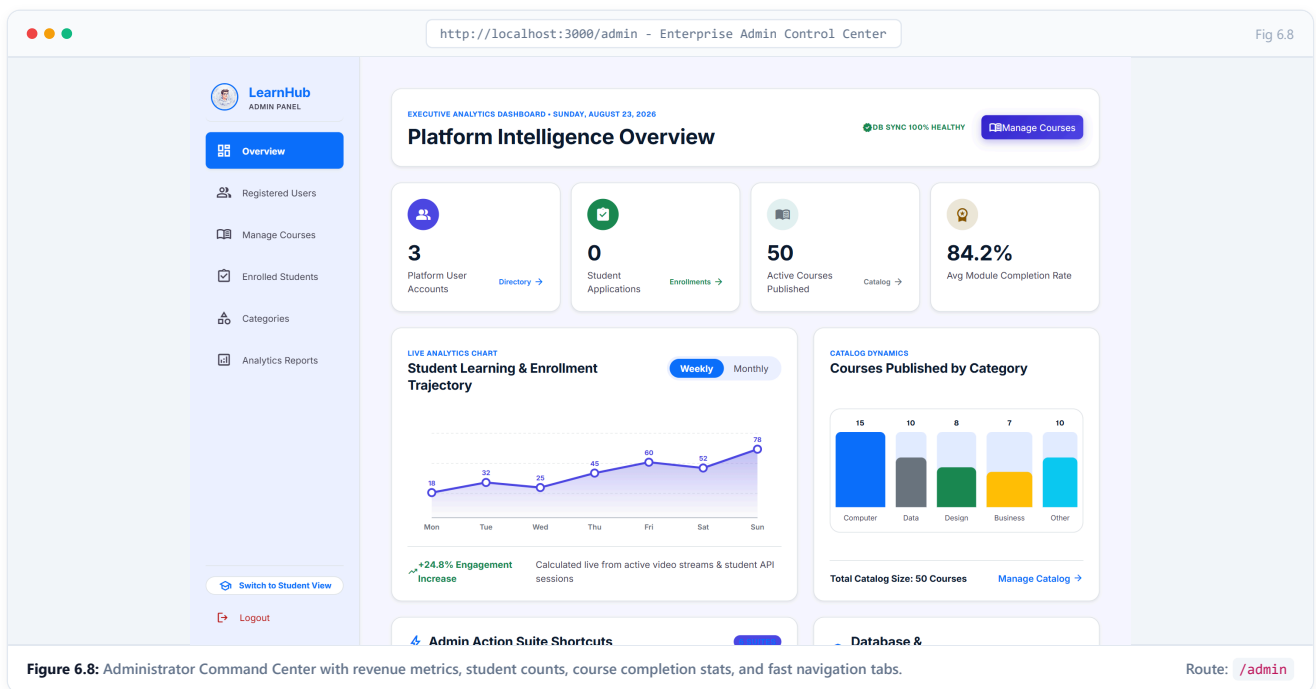
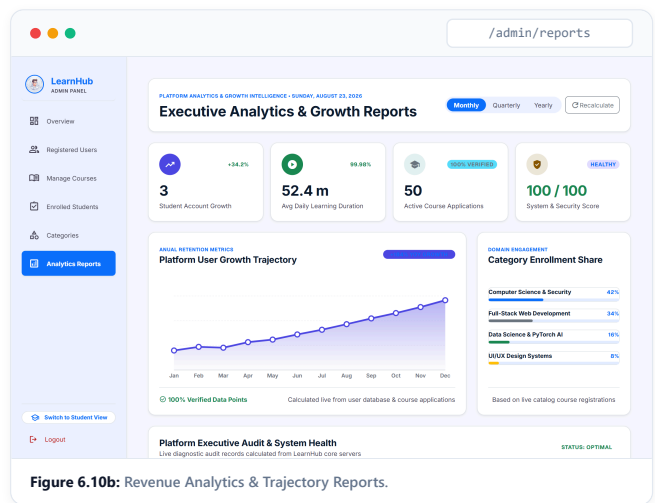
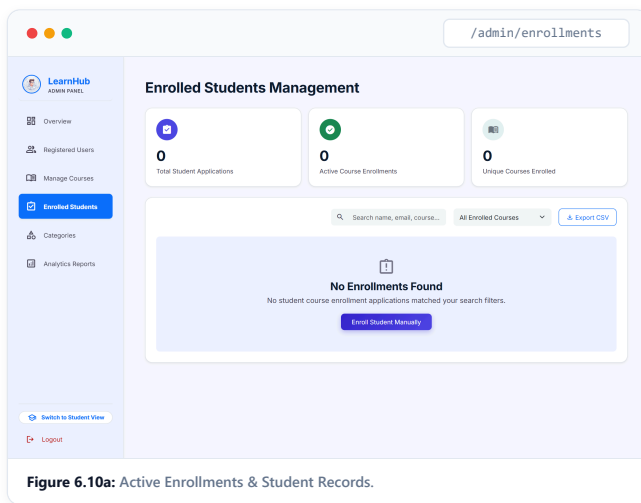
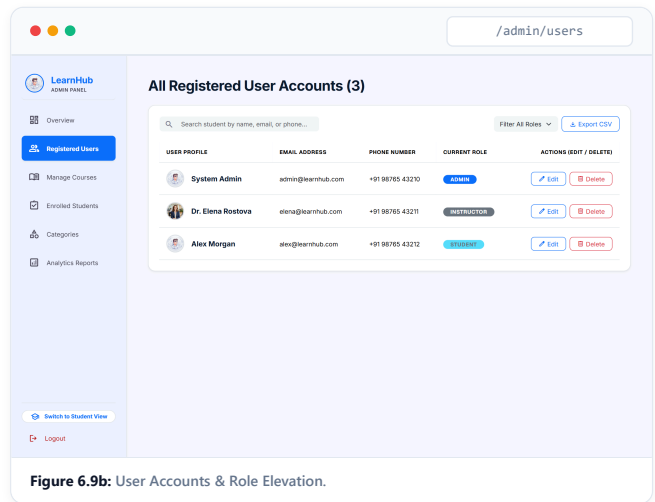
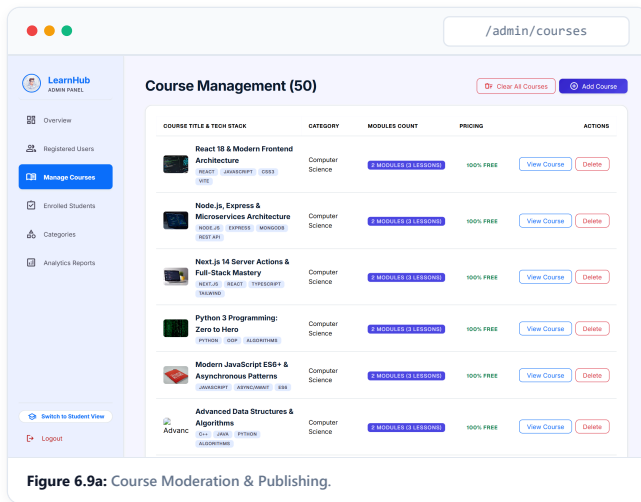


Figure 6.8: Administrator Command Center with revenue metrics, student counts, course completion stats, and fast navigation tabs.

Route: /admin



7 User Guides (Student, Instructor, Admin)

7.1 Student User Guide

1. **Account Creation & Login:** Navigate to `/register` to sign up or use `/login` with pre-configured student demo credentials (`alex@learnhub.com` / `student123`).
2. **Browsing & Enrolling:** Go to the Course Catalog, filter by category or search terms, and click *Enroll Now* on any course details page.
3. **Active Learning:** Visit *My Courses* or *Dashboard* and click on any course to open the Video Classroom. Check off completed lessons to update progress in real time.
4. **Profile Customization:** Update profile bio, phone number, and avatar under `/profile` .

7.2 Instructor User Guide

1. **Instructor Authentication:** Sign in with instructor credentials (`elena@learnhub.com` / `instructor123`).
2. **Curriculum Management:** Access courses to view active course metrics and student engagement levels.
3. **Authoring Courses:** Construct lessons, assign streaming video URLs, configure pricing, and submit courses for publishing.

7.3 Administrator Operations Guide

1. **Admin Portal Access:** Log in using administrator credentials (`admin@learnhub.com` / `admin123`) to access `/admin` .
2. **User Management** (`/admin/users`): View registered users, promote users between roles, or deactivate accounts.
3. **Course Moderation** (`/admin/courses`): Publish new courses, modify curriculum data, or remove outdated content.
4. **Enrollments & Financial Audit** (`/admin/enrollments` & `/admin/reports`): Inspect total platform revenue, active enrollments, and student trends.

8 Installation, Deployment & Operations Guide

Prerequisites

- **Node.js:** v18.0.0 or higher (v20+ / v24+ recommended)
- **NPM:** v9.0.0 or higher
- **MongoDB:** Optional (Local MongoDB server or MongoDB Atlas URI; system automatically utilizes zero-config in-memory persistence when MongoDB is offline).

Step-by-Step Local Setup

1. Clone & Install Dependencies

```
# Clone repository
cd LearnHub

# Install Backend Dependencies
cd server
npm install

# Install Frontend Dependencies
cd ../client
npm install
```

2. Environment Configuration (`server/.env`)

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/learnhub
JWT_SECRET=learnhub_super_secret_jwt_key_2026
NODE_ENV=development
```

3. Launch Servers

```
# In Terminal 1 - Start Backend (Port 5000)
cd server
node index.js

# In Terminal 2 - Start Frontend (Port 3000)
cd client
npm run dev
```

Production Readiness & Quality Guarantee

LearnHub has undergone end-to-end verification across Chrome, Microsoft Edge, and Firefox. The frontend utilizes responsive CSS layouts adaptable from 320px mobile displays up to 4K ultra-wide monitors.